AI 시스템반도체 설계 2기 · VerilogHDL 미니프로젝트 1

Stopwatch · Watch · Timer

Basys-3 한 보드 · FSM + 데이터패스 설계 · 100 MHz Top 시뮬레이션 검증



시계 리셋 화면 (시.분)

목차	발표
1 표지 · 2 개요 · 3 B/D	이종혁
4 모듈: 스톱워치 → 시계 · FND → 타이머	이종혁 → 정안성 → 정보윤
5 시뮬레이션: 스톱워치 → 시계 → 타이머	이종혁 → 정안성 → 정보윤
6 Trouble Shooting ① 디바운서 · ② tick	정보윤
7 결론 · 고찰	정보윤

이종혁 · 정안성 · 정보윤 · 2026-10-19 · v2.0 · 김창형 교수님

소스 · TB: GitHub BEST_Stop_watch_project (verification_v2)

개요 — 세 기능을 같은 틀(FSM + 데이터패스)로 설계 · 검증

결과: TB 13개 · 검사 447개 PASS · LUT 384 · FF 228 · WNS +3.440 ns · 보드 동작 확인

목적

① FSM(언제) · 데이터패스(얼마나) 분리 설계를 기능 3개에 반복 적용

② 버튼 → 디바운서 → FSM → 카운터 → FND 전 경로를 Top 시뮬레이션으로 검증 (100 MHz 그대로)

③ FND 4자리 1개를 세 기능이 공유하는 표시 구조

기능	기본 (교수님 사양)	추가 (팀)
스톱워치	시작/정지 · 지우기 · 업/다운	**랩타임 (D)**
시계	12:00:00.00 시작 · 24시간	**시간 설정 (sw2) · 12/24 (sw3)**
타이머	—	**+10초 · +1분 · +10분 · 알람**
공통	점 0.5 s 점멸 · LED · FND 1개	**버튼 화면 게이팅**

447 / 447
TB 13개 검사 PASS

10.000 ms
1/100초 tick 주기

384 · 228
LUT · FF (1.8% · 0.5%)

+3.440 ns
WNS @100 MHz · 래치 0

표시 범위 스톱워치 00:00:00.00 ~ 23:59:59.99 · 시계 00:00 ~ 23:59 (12시간 표시 1 ~ 12) 타이머 00:00 ~ 59:59

검증 방식 Top 을 넣은 TB 가 버튼 핀을 직접 누르고 FND 자리별 출력까지 자동 판정. 긴 경계(23:59:59 등)는 경계 바로 앞 값을 넣고(초기값 조정) 실제 tick 으로 넘김

구현 숫자: Vivado 2022.2 · Basys-3 xc7a35t · place & route 후 · 팀 저장소 최신 소스 기준

구성 — 스위치로 화면을 고르고, 버튼은 그 화면의 기능에만 들어간다

화면 우선순위: sw4(타이머) > sw0(스톱워치 / 시계) · 다른 화면의 기능도 뒤에서 계속 동작



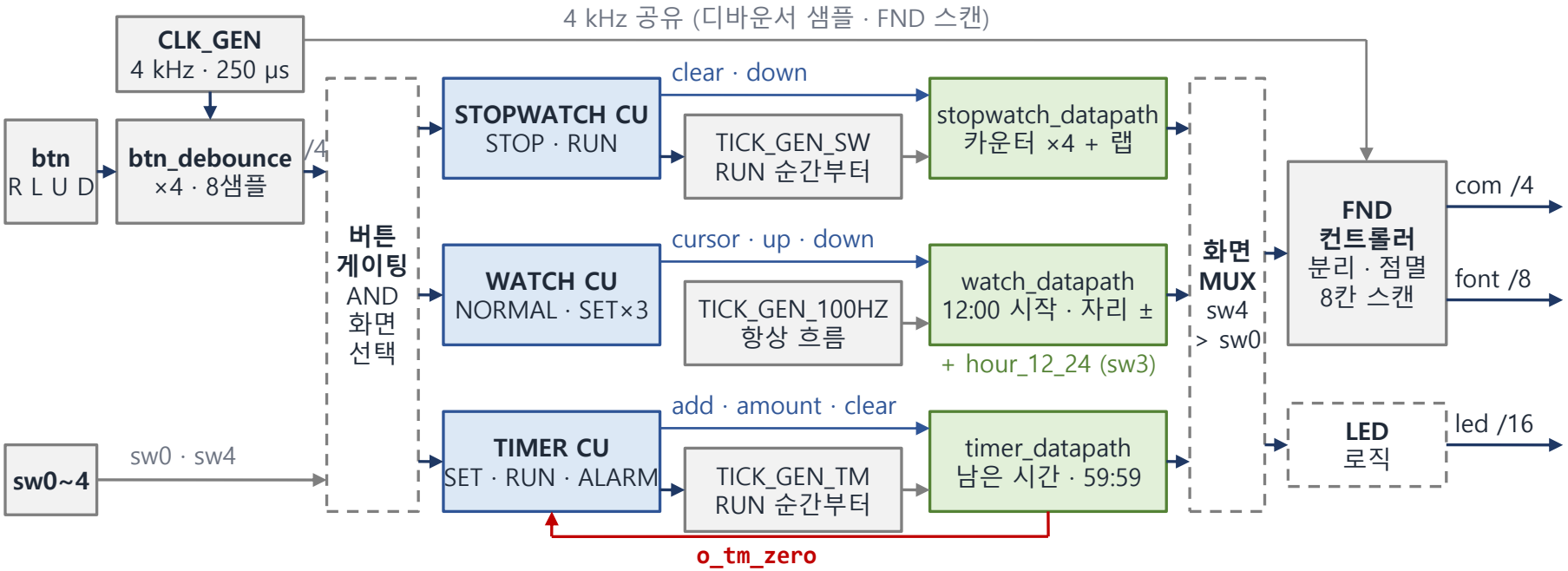
스위치	핀	0	1
sw0	V17	시계	스톱워치
sw1	V16	초.1/100초	시.분
sw2	W16	보통	시계 시간 설정
sw3	W17	12시간 표시	24시간 표시
sw4	W15	—	**타이머 화면 (최우선)**

버튼 (핀)	스톱워치	시계 (sw2 = 1)	타이머
R T17	시작 ↔ 정지	커서 →	+1분
L W19	지우기 (정지 중)	커서 ←	시작 / 취소
U T18	업 ↔ 다운 (정지 중)	커서 자리 +	+10초
D U17	랩 (고정 ↔ 실시간)	커서 자리 –	+10분
C U18	리셋 — 시계 12:00:00.00, 스톱워치 · 타이머 0		

알람 중에는 아무 버튼 = 알람 끄기 · 시계 설정 커서: 분 일 → 분 십 → 시 (양 끝은 반대쪽으로 돌아감)

Block Diagram — 버튼에서 FND 까지

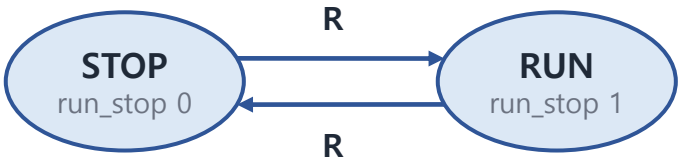
기능마다 컨트롤 유닛(FSM, 언제) + 데이터패스(값) 한 쌍 · 입력 · 박자 · 표시 블록은 셋이 공유



요약 디바운서 4 kHz 8샘플 → 1클럭 · 게이팅: 고른 화면의 FSM 에만 · tick: 기능마다 10 ms · 화면 MUX → FND 1개

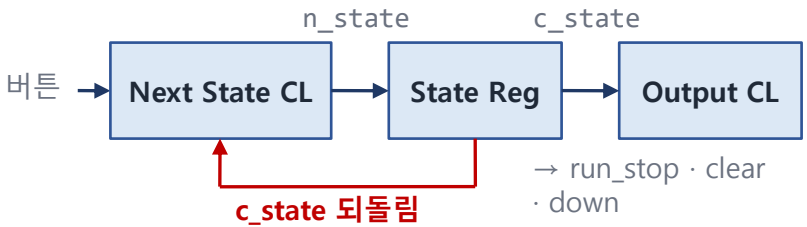
스톱워치 ① 모듈 — FSM 2상태 + 카운터 4단 + 랩 레지스터

FSM 은 "언제 셀지"(run · clear · down)만, 값은 데이터패스 — RUN 중 L · U 는 무시 (실수로 지워지지 않게)

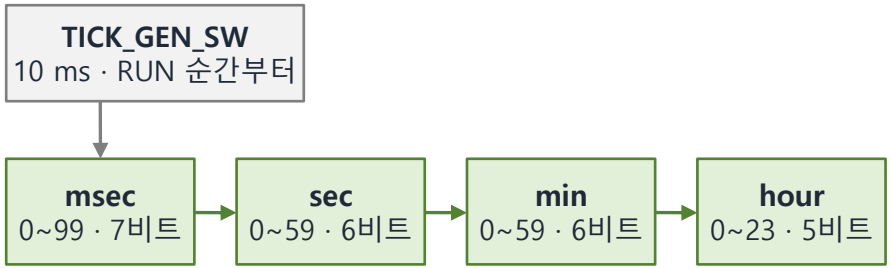


STOP 에서만
L → clear 1클럭 (0 으로)
U → down 반전 (업 ↔ 다운)

RUN 중
L · U 무시 (n_state 그대로)



stopwatch_datapath



o_tick: 한 바퀴마다 윗칸 +1 (다운이면 −1)

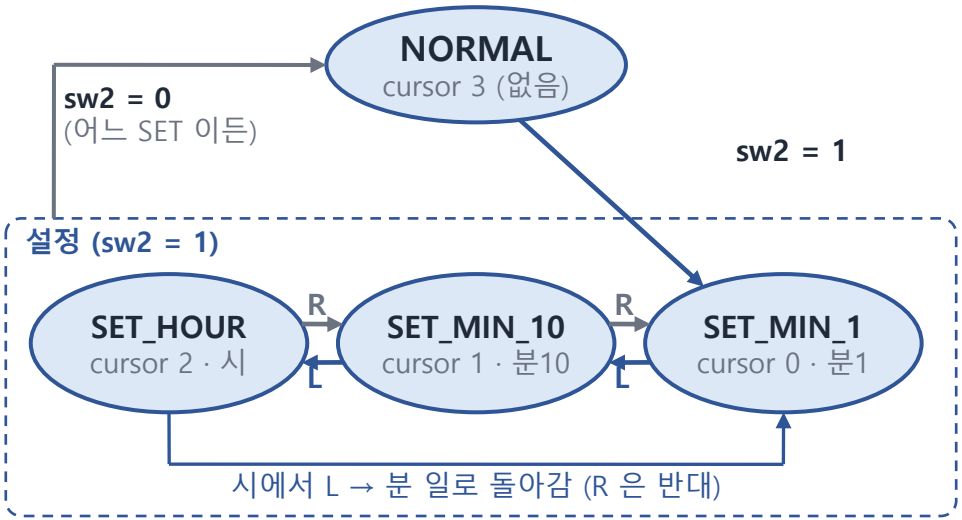
lap_controller
D: 그 순간 값 저장 → 화면 고정 ↔ 실시간 · clear 우선

같은 카운터 4번 재사용 stopwatch_tick_counter #(TIMES, BIT_WIDTH) · 비트 폭 = $clog2(TIMES)

출력 o_sw_msec /7 · sec /6 · min /6 · hour /5 · o_sw_lap → 화면MUX · led7

시계 ① 모듈 — 설정 FSM 4상태 + 12:00 에서 시작하는 카운터

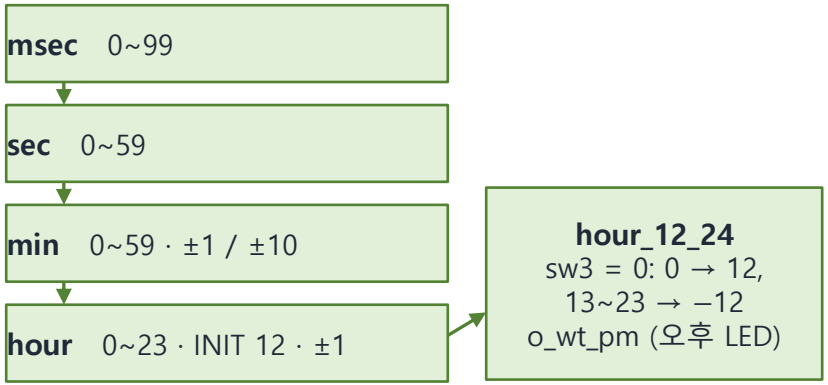
커서 번호 = FND 자리 번호 · 설정은 그 자리만 ± (윗자리 올림 없음) · tick 과 버튼이 같은 클럭이면 tick 우선



U / D (밀리 출력) 상태 그대로, 그 클럭에 o_wt_up / o_wt_down → 커서 자리 ±1 (분 십 ±10)

FSM 구조 Next State · Output CL + State Reg, c_state 되돌림

watch_datapath

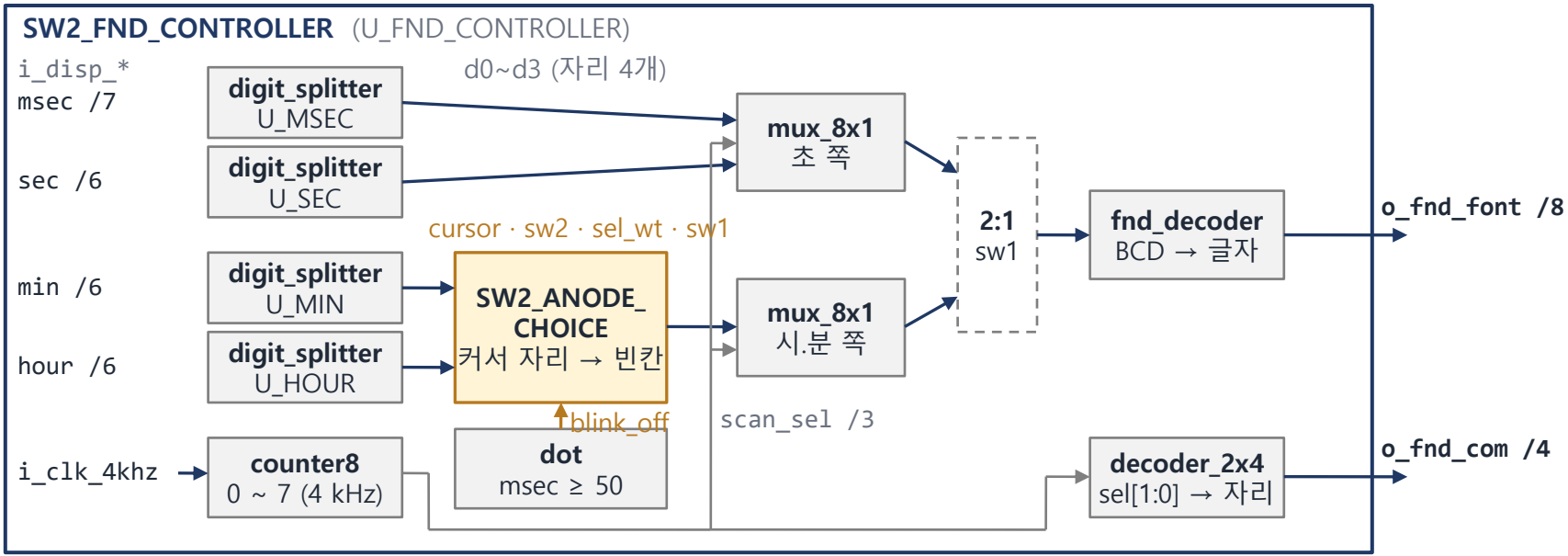


o_tick: 아래 칸 한 바퀴 → 윗칸 +1

watch_tick_counter 수업 카운터 + 리셋값 INIT · 설정 입력(up · down · step10)
설정은 그 자리만 59 +1 → 00 (시 그대로)
tick 우선 tick 과 같은 클럭의 버튼은 반영 안 함

FND 컨트롤러 — 8칸 스캔 + 가운데 점 + 설정 점멸

4자리가 세그먼트 선(a~g, dp)을 공유 → 한 번에 한 자리만 켜고 4 kHz 로 돌림 (한 칸 250 µs, 한 바퀴 2 ms)

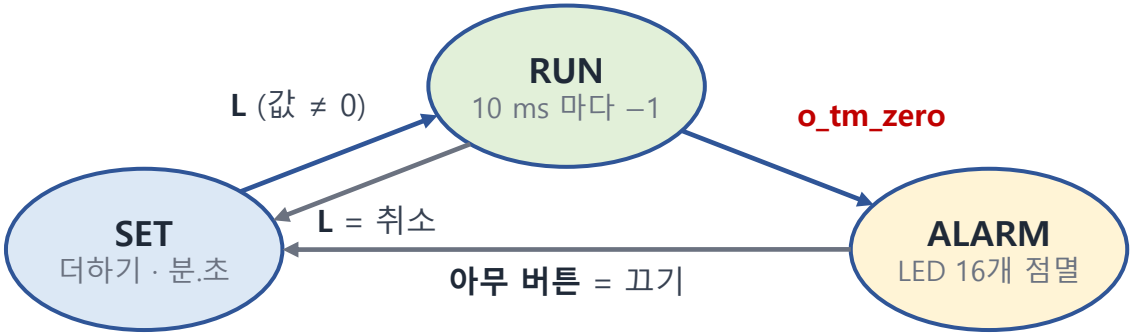


scan_sel	0	1	2	3	4 · 5 · 7	6
fnd_com	1110	1101	1011	0111	—	1011
칸 내용	AN0 숫자	AN1 숫자	AN2 숫자	AN3 숫자	빈칸 F	**AN2 점 E / F**

왜 8칸 점을 숫자와 따로 켜려고 (BCD 남는 코드 E = 점만, F = 끔)
점멸 시계 · sw2 · 시.분 · msec ≥ 50 → 커서 자리 빈칸
10/18 초 쪽 점멸 블록 삭제 (설정은 시.분 화면만, 동작 같음)

타이머 ① 모듈 — SET · RUN · ALARM 3상태 FSM + 남은 시간 데이터패스

0 도달은 데이터패스가 o_tm_zero 로 알림 · 더한 값이 60분 이상이면 59:59 로 제한 (화면 4자리)



timer_datapath

분 · 초 · 1/100초 레지스터
tick 마다 −1, 0 이면 윗자리에서 빌림
add: +10초 · +1분 · +10분 (초 → 분 올림)
min_sum ≥ 60 → 59:59 고정
o_tm_zero = 전부 0

SET 에서	U	R	D	L
하는 일	+10초	+1분	+10분	시작

RUN 중 U · R · D 무시 · 버튼 두 개 이상 같은 클럭 = 모두 무시 (Top, 세 기능 공통)

U_TICK_GEN_TM
RUN 순간부터 10 ms · 취소 · 알람에 0

알람 LED
시계 1/100초 ≥ 50 박자로 16개 0.5 s 점멸

검증 TB 3개 tm_1 더하기 · 상한 21 / tm_2 실제 10초 · 알람 41 / tm_3 취소 · 백그라운드 11 — 모두 PASS

스톱워치 ② 시뮬레이션 — 시작 · 정지 · 지우기, 동작 중 L · U 무시

결과: tb_sw_1 검사 23개 PASS — R → 1.000 s → R 에 1.00 초, RUN 중 L · U 가 와도 값 · 상태 그대로

P	입력	기대 (판정)
2	R → 1.000 s → R	STOP→RUN→STOP, 1.00 초 (tick 100번)
3	120 ms 동작 후 R, 200 ms 대기	STOP 동안 값 그대로
4	정지 중 D(랩) → L	clear 펄스 → 0, 랩 해제
5	**동작 중 L, 그다음 U**	**RUN 유지, clear 없음, 계속 증가**
6	정지 중 R + L 같은 순간	둘 다 무시 · STOP 유지
7	시계 화면에서 R · L · U · D	게이팅 0 → 변화 없음

캡처 자리 S1 — tb_sw_1 P2 · 1.000 s RUN (이종혁)
신호: btn_R · U_BD_R/o_btn · U_STOPWATCH_CONTROL_UNIT/c_state · o_sw_run_stop · U_TICK_GEN_SW/o_tick_10ms · o_sw_msec · o_sw_sec
구간: 0 ~ 1,015 ms 전체 1장 (R 펄스 7.004 ms → STOP→RUN, 1,007.004 ms → RUN→STOP)
확대: 1,006.99 ~ 1,007.02 ms — 100번째 tick 에 msec 99→0 · sec 0→1, 같은 클럭에 R → run_stop 0

캡처 자리 S2 — tb_sw_1 P5 · RUN 중 L · U 무시 (이종혁)
신호: U_BD_L/o_btn · U_BD_U/o_btn · c_state · n_state · o_sw_clear · o_sw_down · o_sw_msec
구간: 1,858 ~ 1,885 ms (L 펄스 1,862.136 ms · U 펄스 1,872.136 ms)
보일 것: c_state RUN 그대로 · o_sw_clear 펄스 없음 · o_sw_down 0 그대로 · msec 30 → 31 → 32 … 계속

스톱워치 ③ 시뮬레이션 — 자정 넘어감 · 다운 빌림 · 랩

결과: tb_sw_2 · tb_sw_3 검사 30개 PASS — 자정 순환 · 다운 빌림 · 랩 중에도 내부는 계속

캡처 자리 S3 — tb_sw_2 P3 · 자정 넘어감 (이종혁)
신호: U_TICK_GEN_SW/o_tick_10ms · o_sw_msec · o_sw_sec · o_sw_min · o_sw_hour
구간: 195 ~ 205 ms (23:59:59.99 → 00:00:00.00 tick ≈ 202.003 ms)
+ ns 확대 1장: 올림이 msec → sec → min → hour 로 한 클럭씩

캡처 자리 S4 — tb_sw_2 P4 · 다운 빌림 (이종혁)
신호: o_sw_down · o_tick_10ms · o_sw_msec · o_sw_sec · o_sw_min · o_sw_hour
구간: 285 ~ 310 ms (00:00:00.00 → 23:59:59.99 → .98)
o_sw_down = 1 이 보이게

캡처 자리 S5 — tb_sw_3 P1 · 랩 (이종혁)
신호: U_BD_D/o_btn · o_sw_lap · led[7] · U_STOPWATCH_DATAPATH/U_LAP_CONTROLLER/i_cnt_msec · o_sw_msec
구간: 200 ~ 320 ms (D 펄스 212.001 ms)
보일 것: o_sw_msec 20 고정 · i_cnt_msec 는 21 → 31 계속 · lap 1 · led7 1

스톱워치 ④ 보드 동작 — 시뮬레이션과 같은 장면

결과: 1분 올림 · 랩 · 동작 중 L 무시 · 0 ↔ 23:59:59.99 를 Basys-3 에서 확인



① 리셋값 00:00:55 (시연용)



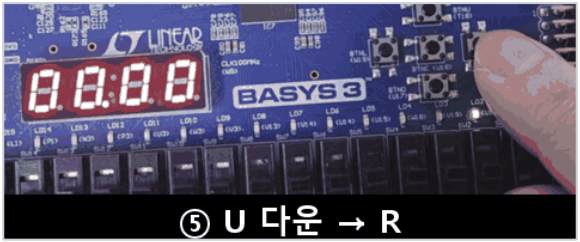
② 59.99 → 00 (초 넘어감)



③ D → 랩: 화면만 고정 (led7)



④ 동작 중 L → 무시 (계속 감)



⑤ U 다운 → R



⑥ U 업 · R → 23.59

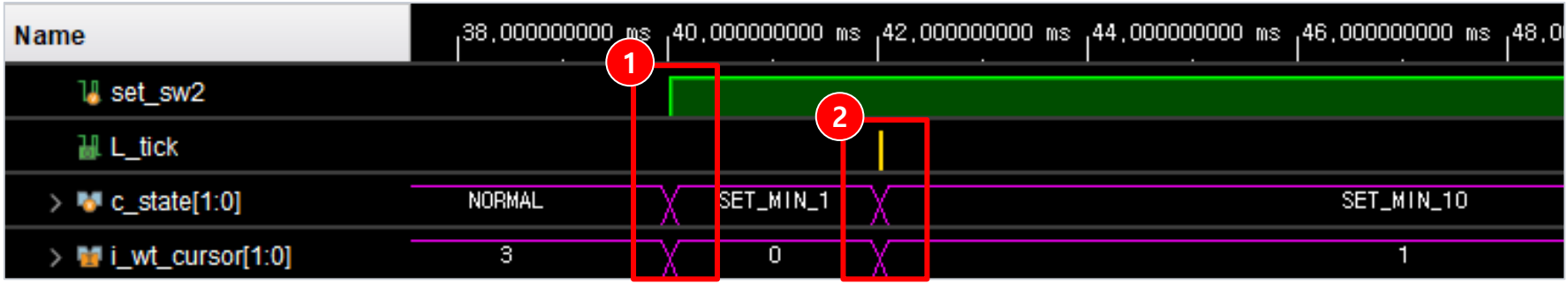
보드에서 본 것

59.99 다음 분 +1 · 랩 중에도 안에서는 계속 셈 · 동작 중 L 무시 · 다운 0 → 23:59:59.99 · 업 23:59:59.99 → 00:00 — 시뮬레이션(9 · 10장) 과 같은 결과

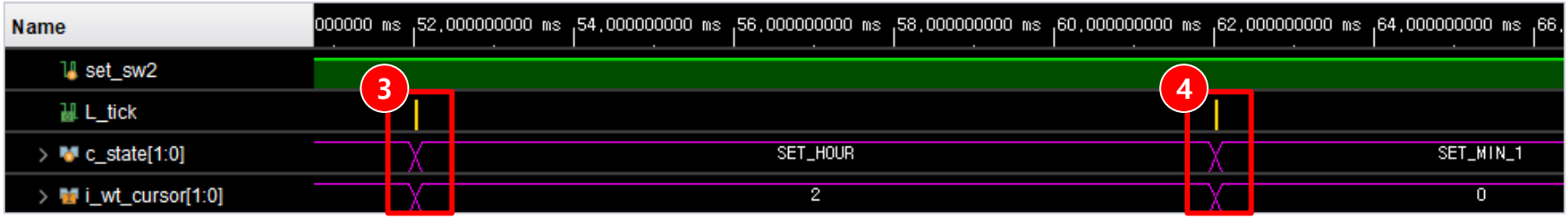
GIF (슬라이드쇼에서 자동 반복) · 시연용 빌드 = 리셋값만 경계 바로 앞 (시계 23:59:50 · 스톱워치 00:00:55)

시계 ② 시뮬레이션 — 설정 진입 · 커서 이동

결과: sw2 = 1 → 분 일(SET_MIN_1) 에서 시작, L 마다 왼쪽 자리로 · 시에서 L → 분 일로 돌아감



① **sw2 ↑ → SET_MIN_1, cursor 0 (분 일)** ② L → SET_MIN_10, cursor 1

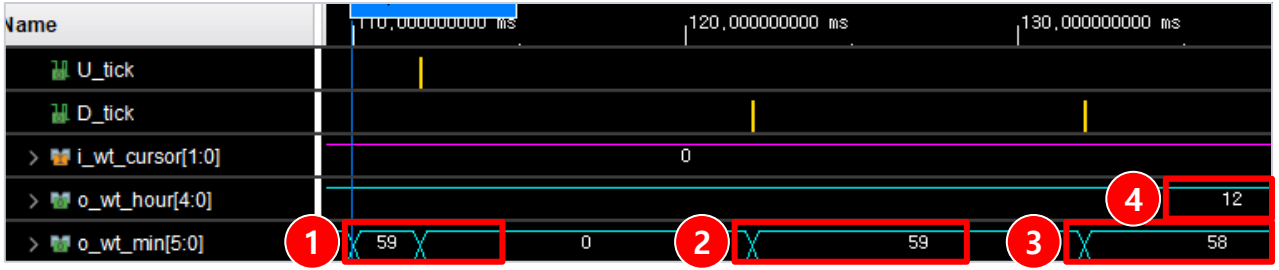


③ L → SET_HOUR, cursor 2 (시) ④ **L → SET_MIN_1, cursor 0 (돌아감)**

tb_wt_1 P3 · 40 ~ 66 ms · 위아래는 이어진 구간 · 캡처: 정안성

시계 ③ 시뮬레이션 — 경계: 설정은 그 자리만 돌고 윗자리는 그대로

결과: 12:59 에서 U → 12:00 (시 12 유지), D → 12:59 · 23:30 에서 시 U → 00:30 = 12시간 표시 12 · 오전

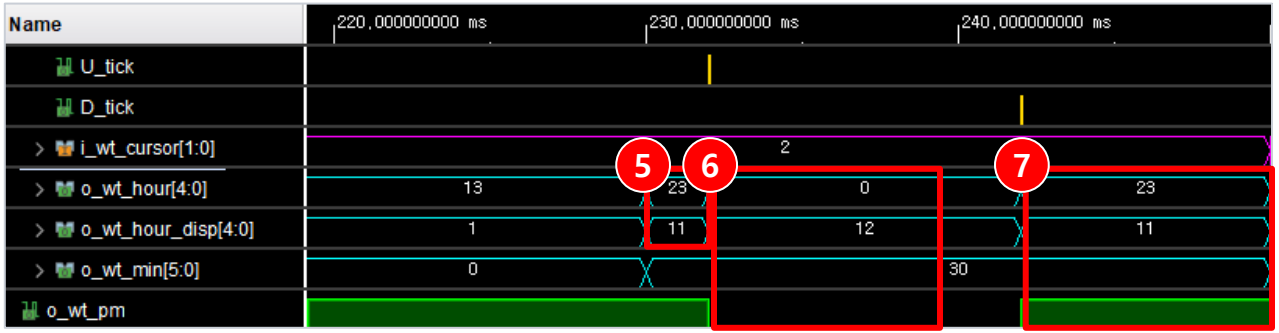


① **U: 59 → 00**

② D: 00 → 59

③ D: 59 → 58

④ **시 12 그대로**



⑤ 23:30 넣음 (표시 11, 오후)

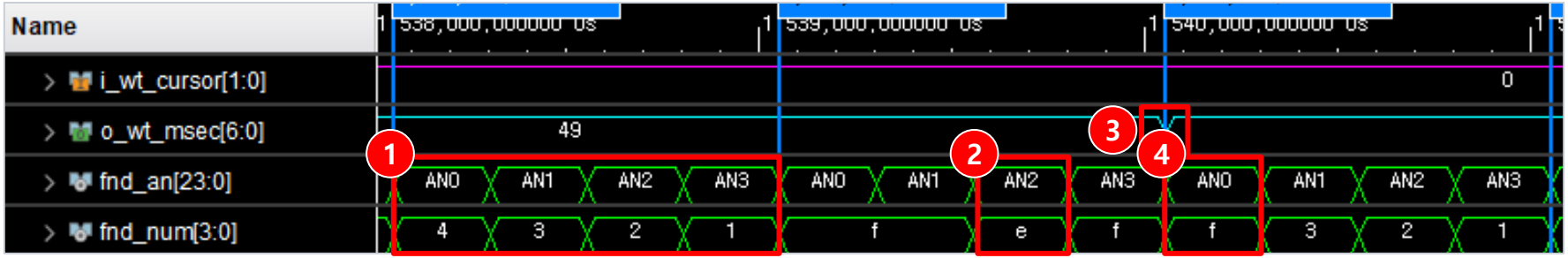
⑥ **U: 시 0 → 표시 12 · 오전 (pm 0)**

⑦ D: 시 23 → 표시 11 · 오후

tb_wt_1 P4 · P6 · 캡처: 정안성

시계 ④ 시뮬레이션 — 설정 점멸이 FND 출력까지

결과: 1/100초 49 → 50 에서 커서 자리(AN0, 분 일)가 빈칸, 가운데 점도 꺼짐 → 0.5 s 마다 반복



① 49: AN0~AN3 = 4 3 2 1 → "12.34" ② AN2 점 칸 = E (점 켬)

③ msec 49 → 50 ④ **AN0 = F (빈칸) → 커서 자리 꺼짐**

FND 읽는 법

fnd_an = 지금 켜진 자리, fnd_num = 그 자리 BCD (fnd_font 로 바뀌기 전) · 한 칸 250 µs · 8칸 = 2 ms

점멸 조건

시계 화면 · sw2 = 1 · 시.분 표시 · 1/100초 ≥ 50 → 커서 자리 = 빈칸 (0.5 s 마다)

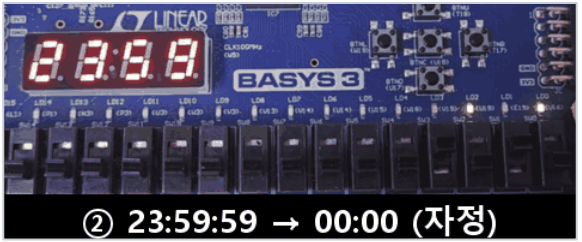
tb_wt_3 · 1,538 ~ 1,542 ms (캡처: 정안성)

시계 ⑤ 보드 동작 — 시뮬레이션과 같은 장면

결과: 자정 넘김 · 12시간 표시 · 설정 점멸 · 자리별 ± 를 Basys-3 에서 확인



① btnC 리셋 → 23:59:50 (시연용)



② 23:59:59 → 00:00 (자정)



③ sw3 12시간 표시: 자정 = 12 (오전)



④ sw2 설정: 커서 자리(분 일) 깜빡



⑤ L · L: 커서 → 시 (깜빡)

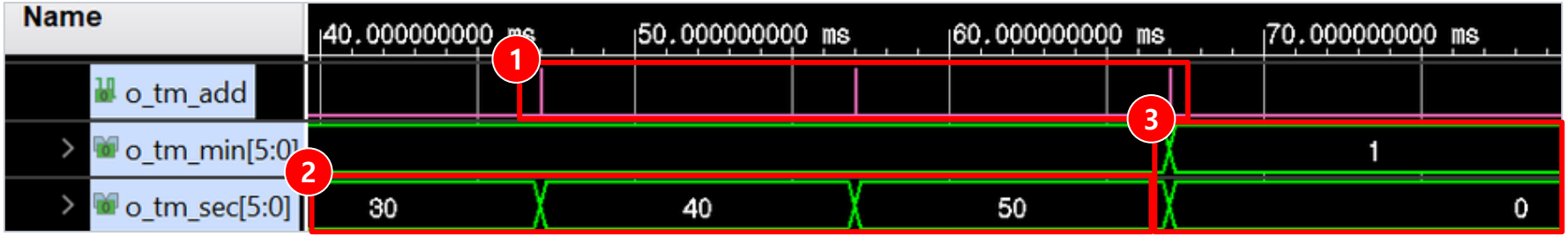
보드에서 본 것

리셋 23:59:50 → 10 초 뒤 자정 · 12시간 표시에서 0시 = 12 · 설정 중 커서 자리 깜빡 · 그 자리만 ± (분 00 → 59 에 시 그대로, 시 00 → 23) — 시뮬레이션(12 ~ 14장) 과 같은 결과

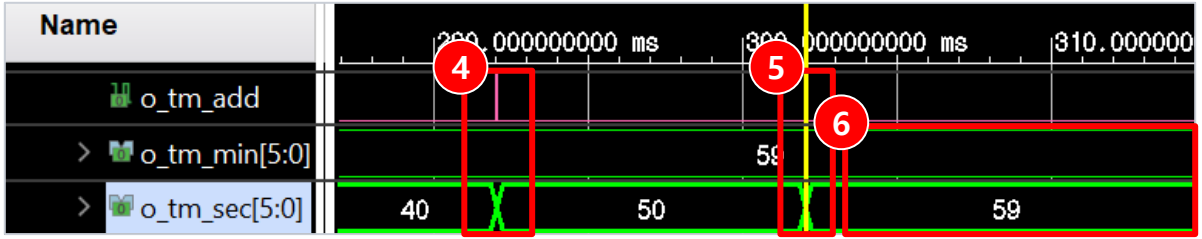
GIF (슬라이드쇼에서 자동 반복) · 시연용 빌드 = 리셋값만 경계 바로 앞 (시계 23:59:50 · 스톱워치 00:00:55)

타이머 ② 시뮬레이션 — 더하기 올림 · 59:59 상한

결과: U 6번째 00:50 + 10초 = 01:00 (60초 아님) · 59:50 + 10초 → 59:59 에서 멈춤



① U 4 · 5 · 6번째 add 펄스 ② 초 30 → 40 → 50 ③ **6번째: 분 1 · 초 0**



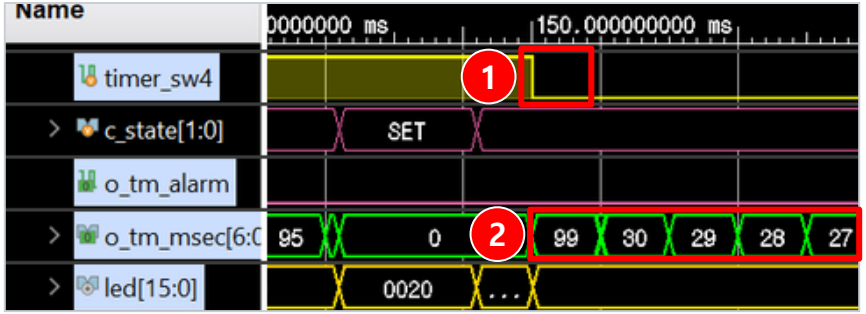
④ U: 59:40 → 59:50

⑤ **U: 59:50 → 59:59**

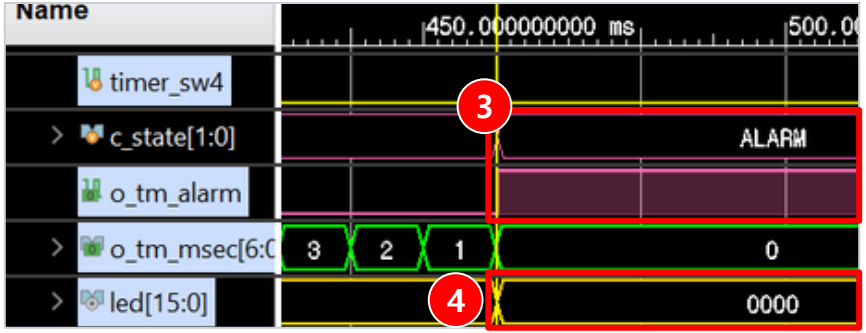
⑥ 59:59 유지 (60:00 아님)

타이머 ③ 시뮬레이션 — 다른 화면에서도 계속 줄고, 0 이면 알람

결과: sw4 를 내려 시계 화면이 돼도 타이머는 감소 → 0 에서 ALARM, LED 16개 0.5 s 점멸



① sw4 1 → 0 (시계 화면) ② 값은 계속 감소



③ **RUN → ALARM** ④ LED 점멸 (이 순간 꺼짐 0000)

알람 LED

시계 1/100초 ≥ 50 → 16개 켬, < 50 → 끔 = 0.5 s 주기 (tb_tm_2 P2 판정)

알람 끄기

아무 버튼 → SET, 값 그대로 (P3) · 다른 화면 버튼으로는 못 끔 (tb_tm_3 P4) · RUN 중 L = 취소 → 00.00 (tb_tm_3 P2)

tb_tm_3 P3 (캡처: 정보윤)

타이머 ④ 보드 동작 — 시뮬레이션과 같은 장면

결과: 0 에서 시작 안 함 · 1분 올림 · 59:59 상한 · 취소 · 다른 화면에서도 알람 을 Basys-3 에서 확인



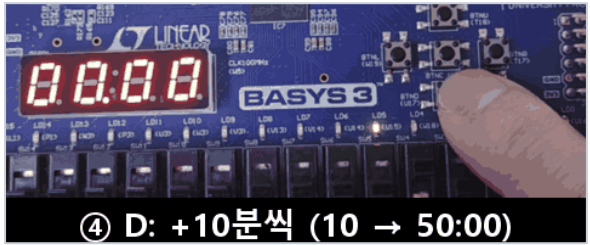
① sw4 → 타이머 화면 (가장 우선)



② U: +10초씩



③ R: +1분씩 → 07:00



④ D: +10분씩 (10 → 50:00)



⑤ L → 시작 (06:59 ...)



⑥ U → L 시작 (00:10)



⑦ 0 → LED 16개 알람 (스톱워치 화면)

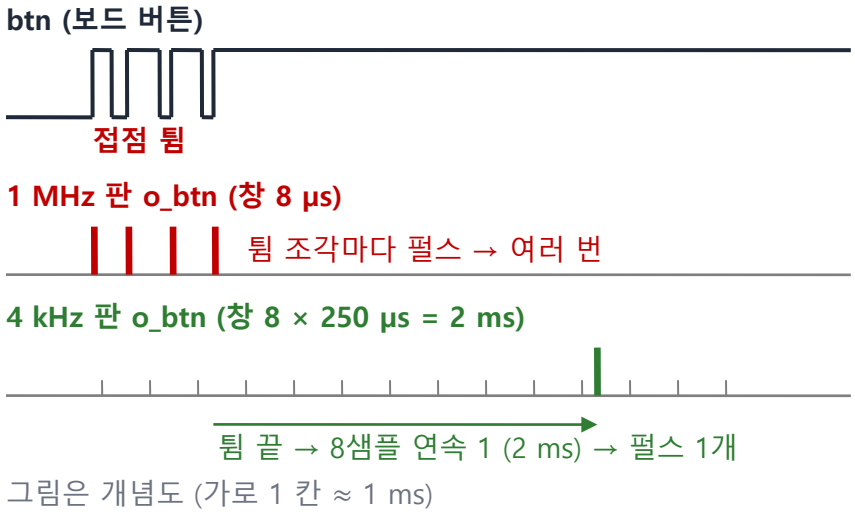
보드에서 본 것

0 에서 시작 안 함 · 60초 = 1분 · 최대 59:59 · 동작 중 L = 취소 · 화면을 바꿔도 계속 줄고 0 이면 어느 화면이든 알람 · 버튼은 알람만 끔
— 시뮬레이션(16 · 17장) 과 같은 결과

GIF (슬라이드쇼에서 자동 반복) · 시연용 빌드 = 리셋값만 경계 바로 앞 (시계 23:59:50 · 스톱워치 00:00:55)

트러블슈팅 ① — 한 번 눌렀는데 두 번 인식 (디바운서)

① 현상: 보드에서 R 한 번에 STOP → RUN → STOP · ② 원인: 샘플 창 8 µs ≪ 접점 튐 (ms 단위)



그림은 개념도 (가로 1 칸 ≈ 1 ms)

③ 해결 — 4 kHz 하나로 두 가지

디바운서

CLK_FRQ 4000 → 250 µs 마다 샘플, 8번 연속 1 = 2 ms 눌려야 인정

FND 스캔 (같은 4 kHz)

8칸 × 250 µs = 한 바퀴 2 ms (500 Hz)

Basys-3 설명서 권장 한 바퀴 1 ~ 16 ms 안 → 깜빡임 없음

숫자 4칸 = 1 ms, 빈칸 · 점 4칸 = 1 ms (자리당 250 µs)

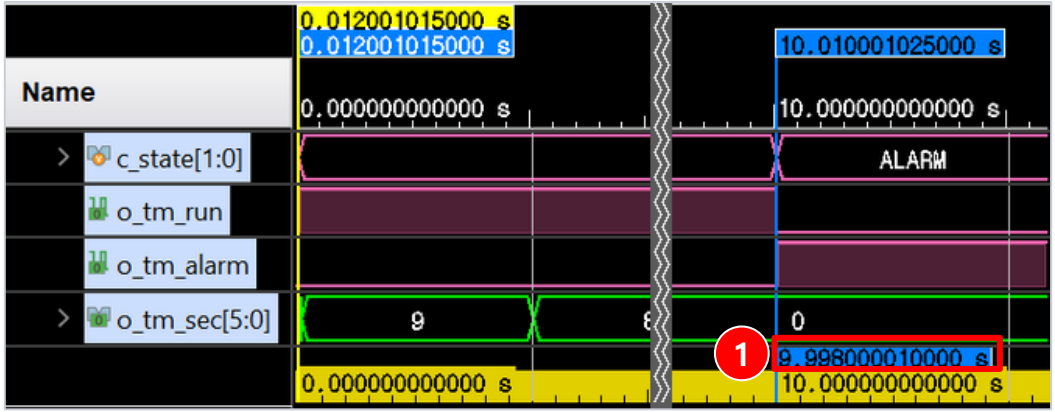
→ 분주기 1개를 디바운서 4개 + FND 가 공유

④ 결과	Before 1 MHz	After 4 kHz
시뮬: 같은 튀는 입력 20번 → o_btn 개수	**158개 (20번 모두 틀림)**	**20개 (모두 1번)**
보드: 타이머 U 20번 → 표시 (기대 03.20)	**측정 필요**	**측정 필요**
대가	반응 8 µs	반응 1.75 ~ 2 ms (못 느낌)

튐 모델: 누를 때 · 뗄 때 20 ~ 400 µs 조각 4 ~ 11개 (실제보다 심함) · Icarus 실행

트러블슈팅 ② — 10초 타이머가 9.998초에 울림 (현상 · 원인)

① 현상: 시작 → 알람 9.998 s · ② 원인: 100 Hz tick 하나를 시계 · 스톱워치 · 타이머가 같이 씀

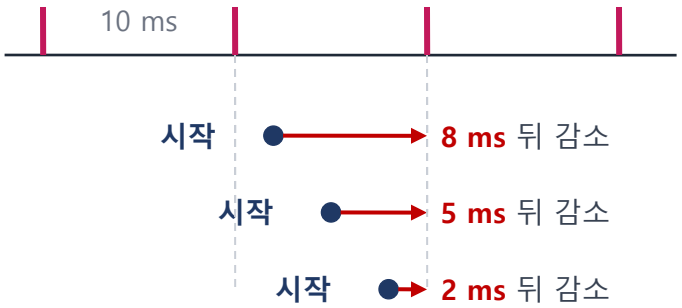


① 현상 RUN 시작 → ALARM 간격 **9.998000010 s**

tb_tm_2 P1 · 10/17 소스 · 가운데 ≋ = 1~9 s 생략 (캡처: 정보윤)

② 원인

공유 tick (리셋부터 항상 10 ms 박자)



시계: 시작이 없어 상관없음

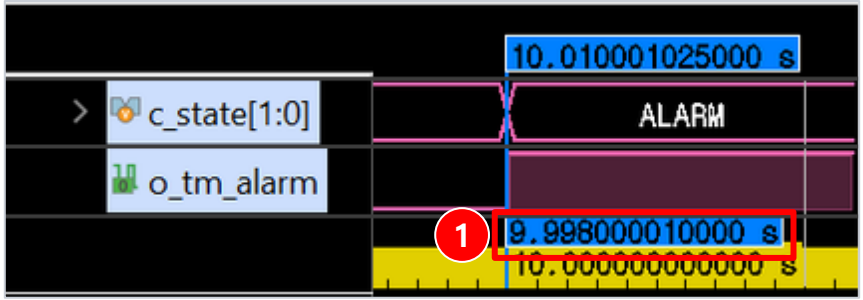
스톱워치 · 타이머: 누른 순간이 0 이어야 함

측정 시작 시각을 3 ms 씩 바꿔 돌린 TB 에서 첫 감소 8 / 5 / 2 ms

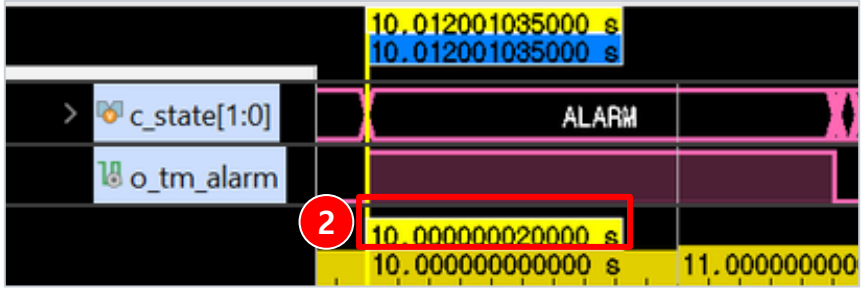
트러블슈팅 ② — 4가지 구조 비교, 100 Hz 만 기능마다 따로

④ 결과: 9.998 s → 10 s + 20 ns (2클럭 고정 지연, 매번 같음) · 대가 FF +42

판	구조	LUT	FF	시작 → 첫 tick
①	수업 방식: 디바운서마다 분주 + 데이터패스마다 tick	415	292	매번 10 ms
②	기존 팀 판: 4 kHz 공유 + 100 Hz 1개 셋이 공유	321	186	**최대 10 ms 일찍**
③	참고: ② + 4 kHz 를 enable 로	311	172	② 와 같음
④	**4 kHz 공유 + 100 Hz 기능마다 (RUN 순간부터)**	**384**	**228**	**매번 정확히 10 ms**



Before 간격 **9.998000010 s**



After 간격 **10.000000020 s**

Vivado 2022.2 · xc7a35t · 구현 후 · 네 판 모두 최종 코드 · 기본 전략 · ①③ 은 비교용 판 · 배치 측정

결론 · 고찰

결론: 목표 기능 전부 Top TB PASS · 보드 확인 / 고찰: 남은 경고 · 입력 처리의 한계와 다음 목표

기능	목표	Top TB	보드
스톱워치	시작 · 정지 · 지우기 · 업다운 · 랩 · 자정	**53 PASS**	**확인**
시계	12:00 시작 · 설정 · 점멸 · 12/24 표시	**264 PASS**	**확인**
타이머	더하기 · 59:59 상한 · 10 s · 알람	**73 PASS**	**확인**
공통	디바운서 · FND 스캔 · 동시 입력 무시	**57 PASS**	**확인**

구현 LUT 384 (1.8%) · FF 228 (0.5%) · WNS +3.440 ns @100 MHz · 래치 0
1/100초 = 10.000 ms · 10초 타이머 = 10 s + 20 ns

결론 FSM(언제) + 데이터패스(얼마나) 분리로 세 기능을 같은 구조로 만들고, 버튼 → FND 전체 경로를 100 MHz 그대로 시뮬레이션해 목표 기능이 모두 사양대로 동작함을 확인

고찰 (아쉬운 점 → 다음 목표)

① 경고 35개 남음

4 kHz 펄스를 클럭으로 씀 (TIMING-17) → enable 방식이면 0 개 (비교용 판 확인) → 다음 프로젝트 적용

② 디바운서 한계

8번 연속 1 만 봄 → 누른 채 잠깐 끊기면 두 번 인식 가능 → 뗄 때도 8번 연속 0 보기 검토

③ 긴 경계

23:59:59 등은 초기값으로 확인 → 실제 24시간 흐름은 보드로만

부록 A — 검증 전체: TB 13개 · 검사 447개 PASS

Top 을 넣은 TB · 100 MHz 그대로 · 콘솔 [SUMMARY] 자동 판정 · 최신 소스로 다시 실행

폴더	TB	확인하는 것	검사	결과	시뮬 시간
top	tb_top_1	디바운서: 누름 시간 · 바운스	24	PASS	2,205 ms
top	tb_top_2	FND 스캔 · 자리별 출력 · 점	22	PASS	2,180 ms
top	tb_top_3	버튼 동시 입력: 세 기능 모두 무시 (10/18)	11	PASS	302 ms
스톱워치	tb_sw_1	시작 · 정지 · 지우기 · 동작 중 무시 · 동시 R+L	23	PASS	2,290 ms
스톱워치	tb_sw_2	올림 · 자정 순환 · 다운 빌림	15	PASS	485 ms
스톱워치	tb_sw_3	랩 고정 · 해제 · 지우기	15	PASS	1,230 ms
타이머	tb_tm_1	더하기 · 올림 · 59:59 상한	21	PASS	455 ms
타이머	tb_tm_2	실제 10 초 · 알람 · 빌림	41	PASS	11,597 ms
타이머	tb_tm_3	취소 · 백그라운드 · 화면 우선순위	11	PASS	547 ms
시계	tb_wt_1	설정 UI · 커서 · 경계	38	PASS	870 ms
시계	tb_wt_2	1초 정확도 · 자정 올림	13	PASS	2,245 ms
시계	tb_wt_3	12/24 표시 · 점멸 · LED	118	PASS	6,050 ms
시계	tb_wt_4	설정 중 R · L · U · D 동시 누름 16가지	95	PASS	480 ms

최종 코드(동시 입력 무시 포함)로 13개 전부 재실행 · xsim 배치 (, 10/18)

부록 B — 버튼 두 개를 같은 순간에 누르면? → 모두 무시

같은 클럭에 버튼 펄스 2개 이상 → 그 클럭 입력 모두 버림 (세 기능 같은 규칙)

화면 · 상태	같은 클럭에 동시 입력	전 (기능마다 우선순위)	지금 (10/18)
스톱워치 STOP	R + L	RUN 시작 + 0 으로 지움	**무시 · STOP · 값 유지**
시계 설정	L + R	L 우선 (커서 왼쪽)	**무시 · 커서 그대로**
시계 설정	U + D	무시 (10/18 오전 수정)	**무시 · 값 그대로**
타이머 SET	U + R	U 만 (+10초)	**무시 · 00:00 그대로**
타이머 SET	L + D	L 만 (시작)	**무시 · 시작 안 함**
버튼 하나만	L · U 등	보통 동작	보통 동작 (그대로)

어디서 Top 의 버튼 게이팅: w_btn_multi = 펄스 두 개 이상 → 12개 게이팅 AND 에 ~w_btn_multi (컨트롤 유닛 · 랩까지 한 번에)
확인 tb_top_3 (세 기능 동시 입력 · 검사 11개) · tb_sw_1 P6 · tb_wt_4 (시계 16가지) — 고치기 전 소스로 돌리면 tb_top_3 FAIL 7개

같은 클럭이 되는 조건: 디바운서 4 kHz(250 µs) 샘플 → 두 버튼이 250 µs 안에 같이 안정될 때만

부록 C — 합성 · 구현 전략 비교 (PPA) → 기본값 유지

자원 1.8% · WNS +3.44 ns · 0.106 W 셋 다 여유 → 기본값 유지 (Area 는 LUT −13.5% ↔ WNS −0.84 ns)

전략 (Vivado 2022.2)	LUT	FF	WNS @ 100 MHz	전력 (추정)
기본값 (Vivado Defaults) — 채택	**384**	**228**	**+3.440 ns**	**0.106 W**
합성 Area (Flow_AreaOptimized_high)	332 (−13.5%)	227	+2.600 ns	0.115 W
합성 Area + 구현 Area_Explore	336	227	+2.011 ns	0.114 W
성능 (PerfOptimized_high + Performance_Explore)	384	228	+3.440 ns	0.106 W
전력 (Power_DefaultOpt, power_opt_design)	390	228	+3.733 ns	0.104 W

왜 기본값 전략은 빡빡한 제약 쪽을 고르는 것 — 이 설계는 칩의 1.8%, 타이밍 실패 경로 0, USB 전원이라 셋 다 제약이 없음. Area 를 고르면 얻는 것 없이 타이밍 여유만 줄고, Power 효과는 추정 오차 안 (전체 0.106 W 중 0.072 W 는 칩 누설)
네 판 모두 같은 경향 트러블슈팅 ② 표의 ① ~ ④ 에 같은 전략을 적용해도 Performance 는 전부 기본값과 동일, 합성 Area 는 LUT −7 / −28 / −26 / −52 에 WNS 전부 감소, Power 는 0 ~ −0.002 W

최종 코드 · 같은 xdc · xc7a35t · 구현 후 · 전력은 Vivado 추정(신뢰도 Low) · 측정은 배치 실행 (design_prototype/ppa_v3)